โครงงานการจัดทำโปรแกรม

ระบบ POS สำหรับร้านอาหาร

นาย มนัสวิน สะโรธร

รหัสนิสิต 68160101

เสนอ

อาจารย์พีระศักดิ์ เพียรประสิทธิ์

โครงงานนี้เป็นส่วนหนึ่งของรายวิชา

88812265 การสร้างแบบจำลองและการโปรแกรมเชิงวัตถุ

ภาคการศึกษาที่ 2 ปีการศึกษา 2568

สาขาวิชาวิศวกรรมซอฟต์แวร์

คณะวิทยาการสารสนเทศ มหาวิทยาลัยบูรพา

สารบัญ

บทที่ 1 บทนำ..... 1

1.1 วัตถุประสงค์ 1

บทที่ 2 ประโยชน์ที่คาดว่าจะได้รับ 1

บทที่ 3 การออกแบบโปรแกรม..... 2

3.1 ภาพรวมการทำงาน 2

3.2 การออกแบบคลาส 2

บทที่ 4 การออกแบบการทำงานของโปรแกรม 11

4.1 ภาพรวมการทำงานของโปรแกรม..... 11

4.2 การออกแบบขั้นตอนการเปิดโต๊ะ 11

4.3 การออกแบบขั้นตอนการคำนวณยอดรวม 11

บทที่ 5 ผลลัพธ์ของโปรแกรม..... 12

บทที่ 6 สรุปผล..... 15

บทที่ 7 ซอร์สโค้ด 15

7.1 Printable.java – อินเทอร์เฟซสำหรับการพิมพ์รายละเอียด 15

7.2 Person.java – คลาสพื้นฐานสำหรับบุคคล 15

7.3 Staff.java – คลาสพนักงาน..... 16

7.4 Customer.java – คลาสลูกค้า 17

7.5 MenuItem.java – คลาสพื้นฐานสำหรับรายการเมนู..... 17

7.6 Food.java – คลาสอาหาร 18

7.7 Beverage.java – คลาสเครื่องดื่ม 19

7.8 Order.java – คลาสออร์เดอร์..... 20

7.9 Payment.java – คลาสการชำระเงิน 22

7.10 Table.java – คลาสโต๊ะ..... 23

7.11 SimplePOS_GUI.java - คลาส GUI หลัก 25

สารบัญตาราง

ตารางที่ 1 คลาสทั้งหมดของโปรแกรม POS 2
ตารางที่ 2 รายละเอียดของอินเทอร์เฟซ Printable 4
ตารางที่ 3 รายละเอียดของคลาส Person 4
ตารางที่ 4 รายละเอียดของคลาส Staff 5
ตารางที่ 5 รายละเอียดของคลาส Customer 5
ตารางที่ 6 รายละเอียดของคลาส MenuItem 6
ตารางที่ 7 รายละเอียดของคลาส Food 6
ตารางที่ 8 รายละเอียดของคลาส Beverage 7
ตารางที่ 9 รายละเอียดของคลาส Order 8
ตารางที่ 10 รายละเอียดของคลาส Payment 9
ตารางที่ 11 รายละเอียดของคลาส Table 10

สารบัญรูปภาพ

ภาพที่ 1 แผนภาพคลาสโปรแกรม POS 3
ภาพที่ 2 แสดงรายการอาหารและเครื่องดื่ม 12
ภาพที่ 3 แสดงสถานะโต๊ะทั้งหมด 12
ภาพที่ 4 เปิดโต๊ะสำหรับรับลูกค้า 13
ภาพที่ 5 เพิ่มรายการลงในออร์เดอร์ 13
ภาพที่ 6 ชำระเงินและพิมพ์ใบเสร็จ 14
ภาพที่ 7 ตัวอย่างใบเสร็จ 14

บทที่ 1 บทนำ

ในปัจจุบันร้านอาหารมีการแข่งขันสูง การให้บริการที่รวดเร็วและแม่นยำเป็นปัจจัยสำคัญในการดำเนินธุรกิจ ระบบ Point of Sale (POS) เป็นเครื่องมือที่ช่วยจัดการการสั่งอาหาร การคิดเงิน และการจัดการโต๊ะได้อย่างมีประสิทธิภาพ

โครงงานนี้มีวัตถุประสงค์เพื่อพัฒนาโปรแกรม POS สำหรับร้านอาหารโดยใช้หลักการโปรแกรมเชิงวัตถุ โปรแกรมสามารถจัดการโต๊ะ รับออร์เดอร์ คำนวณราคา พร้อมภาษีมูลค่าเพิ่ม และพิมพ์ใบเสร็จได้

1.1 วัตถุประสงค์

1. เพื่อพัฒนาโปรแกรม POS สำหรับร้านอาหารโดยใช้หลักการโปรแกรมเชิงวัตถุ
2. เพื่อศึกษาและประยุกต์ใช้คุณสมบัติการสืบทอด (Inheritance) การห่อหุ้ม (Encapsulation) และการพ้องรูป (Polymorphism)
3. เพื่อสร้างระบบที่สามารถจัดการโต๊ะ รับออร์เดอร์ และคำนวณการชำระเงินได้อย่างมีประสิทธิภาพ

บทที่ 2 ประโยชน์ที่คาดว่าจะได้รับ

1. ได้เรียนรู้การออกแบบและพัฒนาโปรแกรมเชิงวัตถุด้วยภาษา Java
2. ได้เข้าใจหลักการสืบทอด อินเทอร์เฟซ และการพ้องรูป
3. ได้ระบบ POS ที่สามารถนำไปใช้งานจริงในร้านอาหารได้
4. ได้เรียนรู้การสร้าง GUI ด้วย Java Swing

บทที่ 3 การออกแบบโปรแกรม

3.1 ภาพรวมการทำงาน

การออกแบบฟังก์ชันการทำงานของโปรแกรม POS นี้จะกำหนดให้ผู้ใช้งานโปรแกรมคือพนักงานเสิร์ฟ (Staff) โดยที่พนักงานสามารถใช้งานโปรแกรมได้ดังนี้

- เปิดโต๊ะสำหรับรับลูกค้า
- เพิ่มรายการอาหารและเครื่องดื่มลงในออร์เดอร์
- ดูสถานะโต๊ะทั้งหมด
- ชำระเงินและพิมพ์ใบเสร็จ

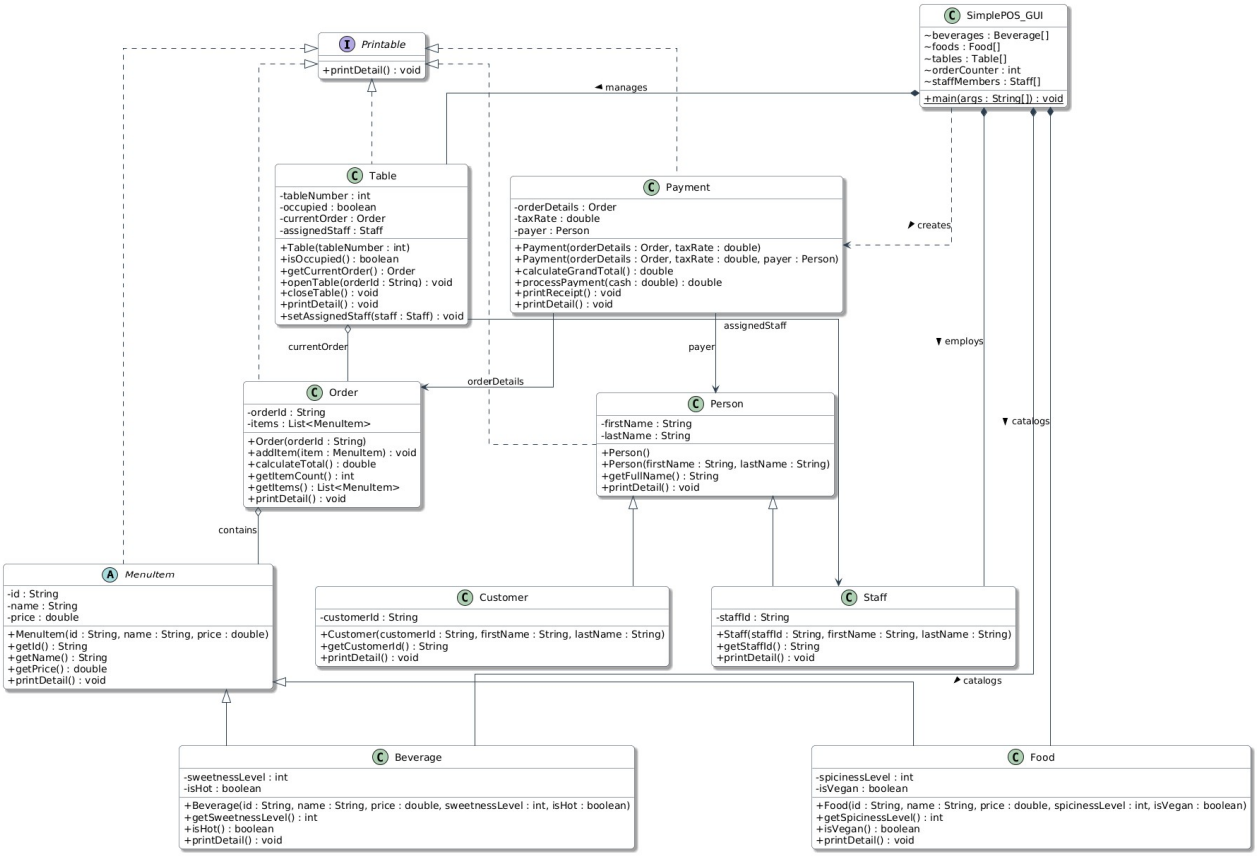
3.2 การออกแบบคลาส

จากการวิเคราะห์และออกแบบโปรแกรม POS นี้ คลาสและเมธอดหลักๆ ของโปรแกรมจะมีคลาสต่างๆ ดังนี้

ตารางที่ 1 คลาสทั้งหมดของโปรแกรม POS

Class	**Base Class**	รายละเอียด
Printable	Interface	อินเทอร์เฟซสำหรับพิมพ์รายละเอียด
Person	ไม่มี	คลาสบุคคลพื้นฐาน
Staff	Person	คลาสพนักงาน สืบทอดจาก Person
Customer	Person	คลาสลูกค้า สืบทอดจาก Person
MenuItem	ไม่มี	คลาสรายการเมนูพื้นฐาน
Food	MenuItem	คลาสอาหาร สืบทอดจาก MenuItem

Class	Base Class	รายละเอียด
Beverage	MenuItem	คลาสเครื่องดื่ม สืบทอดจาก MenuItem
Order	ไม่มี	คลาสออร์เดอร์
Payment	ไม่มี	คลาสการชำระเงิน
Table	ไม่มี	คลาสโต๊ะในร้านอาหาร



ภาพที่ 1 แผนภาพคลาสโปรแกรม POS

รายละเอียดของแต่ละคลาส

ตารางที่ 2 รายละเอียดของอินเทอร์เฟซ Printable

สมาชิกของคลาส	ประเภท	รายละเอียด
void printDetail()	เมธอด	พิมพ์รายละเอียดของวัตถุ

ตารางที่ 3 รายละเอียดของคลาส Person

สมาชิกของคลาส	ประเภท	รายละเอียด
String firstName	แอตทริบิวต์	ชื่อ
String lastName	แอตทริบิวต์	นามสกุล
Person()	คอนสตรัคเตอร์	กำหนดค่าเริ่มต้น
Person(String, String)	คอนสตรัคเตอร์	กำหนดค่าตามพารามิเตอร์
void setFullName(String, String)	เมธอด	กำหนดชื่อและนามสกุลใหม่
String getFirstName()	เมธอด	คืนค่าชื่อ
String getLastName()	เมธอด	คืนค่านามสกุล
String getFullName()	เมธอด	คืนค่าชื่อเต็ม
void printDetail()	เมธอด	พิมพ์ชื่อเต็ม

ตารางที่ 4 รายละเอียดของคลาส Staff

สมาชิกของคลาส	ประเภท	รายละเอียด
String staffId	แอตทริบิวต์	รหัสพนักงาน
Staff(String, String, String)	คอนสตรัคเตอร์	กำหนดค่าตามพารามิเตอร์
String getStaffId()	เมธอด	คืนค่ารหัสพนักงาน
void printDetail()	เมธอด	พิมพ์รายละเอียดพนักงาน

ตารางที่ 5 รายละเอียดของคลาส Customer

สมาชิกของคลาส	ประเภท	รายละเอียด
String customerId	แอตทริบิวต์	รหัสลูกค้า
Customer(String, String, String)	คอนสตรัคเตอร์	กำหนดค่าตามพารามิเตอร์
String getCustomerId()	เมธอด	คืนค่ารหัสลูกค้า
void printDetail()	เมธอด	พิมพ์รายละเอียดลูกค้า

ตารางที่ 6 รายละเอียดของคลาส MenuItem

สมาชิกของคลาส	ประเภท	รายละเอียด
String id	แอตทริบิวต์	รหัสรายการ
String name	แอตทริบิวต์	ชื่อรายการ
double price	แอตทริบิวต์	ราคา
MenuItem(String, String, double)	คอนสตรัคเตอร์	กำหนดค่าตามพารามิเตอร์
String getId()	เมธอด	คืนค่ารหัส
String getName()	เมธอด	คืนค่าชื่อ
double getPrice()	เมธอด	คืนค่าราคา
void printDetail()	เมธอด	พิมพ์รายละเอียด

ตารางที่ 7 รายละเอียดของคลาส Food

สมาชิกของคลาส	ประเภท	รายละเอียด
int spicinessLevel	แอตทริบิวต์	ระดับความเผ็ด (0-100)
boolean isVegan	แอตทริบิวต์	เป็นมังสวิรัติหรือไม่

สมาชิกของคลาส	ประเภท	รายละเอียด
Food(String, String, double, int, boolean)	คอนสตรัคเตอร์	กำหนดค่าตามพารามิเตอร์
int getSpicinessLevel()	เมธอด	คืนค่าระดับความเผ็ด
boolean isVegan()	เมธอด	คืนค่าสถานะมังสวิรัติ
void printDetail()	เมธอด	พิมพ์รายละเอียดอาหาร

ตารางที่ 8 รายละเอียดของคลาส Beverage

สมาชิกของคลาส	ประเภท	รายละเอียด
int sweetnessLevel	แอตทริบิวต์	ระดับความหวาน (0-100)
boolean isHot	แอตทริบิวต์	เสิร์ฟร้อนหรือไม่
Beverage(String, String, double, int, boolean)	คอนสตรัคเตอร์	กำหนดค่าตามพารามิเตอร์
int getSweetnessLevel()	เมธอด	คืนค่าระดับความหวาน
boolean isHot()	เมธอด	คืนค่าสถานะร้อน/เย็น
void printDetail()	เมธอด	พิมพ์รายละเอียดเครื่องดื่ม

ตารางที่ 9 รายละเอียดของคลาส Order

สมาชิกของคลาส	ประเภท	รายละเอียด
String orderId	แอตทริบิวต์	รหัสออร์เดอร์
List<MenuItem> items	แอตทริบิวต์	รายการเมนูในออร์เดอร์
Order(String)	คอนสตรัคเตอร์	กำหนดค่าตามพารามิเตอร์
String getOrderId()	เมธอด	คืนค่ารหัสออร์เดอร์
List<MenuItem> getItems()	เมธอด	คืนรายการเมนูทั้งหมด
int getItemCount()	เมธอด	คืนจำนวนรายการ
void addItem(MenuItem)	เมธอด	เพิ่มรายการในออร์เดอร์
double calculateTotal()	เมธอด	คำนวณยอดรวมย่อย
void printDetail()	เมธอด	พิมพ์รายละเอียดออร์เดอร์

ตารางที่ 10 รายละเอียดของคลาส Payment

สมาชิกของคลาส	ประเภท	รายละเอียด
Order orderDetails	แอตทริบิวต์	ออร์เดอร์ที่ชำระ
double taxRate	แอตทริบิวต์	อัตราภาษี
Person payer	แอตทริบิวต์	ผู้ชำระเงิน
Payment(Order, double)	คอนสตรัคเตอร์	กำหนดค่าตามพารามิเตอร์
Payment(Order, double, Person)	คอนสตรัคเตอร์	กำหนดค่าตามพารามิเตอร์พร้อมผู้ชำระเงิน
Order getOrderDetails()	เมธอด	คืนค่าออร์เดอร์ที่ชำระ
double getTaxRate()	เมธอด	คืนค่าอัตราภาษี
Person getPayer()	เมธอด	คืนค่าผู้ชำระเงิน
double calculateGrandTotal()	เมธอด	คำนวณยอดรวมทั้งหมดรวมภาษี
double processPayment(double)	เมธอด	ประมวลผลการชำระเงิน
void printReceipt()	เมธอด	พิมพ์ใบเสร็จ
void printDetail()	เมธอด	พิมพ์รายละเอียดการชำระเงิน

ตารางที่ 11 รายละเอียดของคลาส Table

สมาชิกของคลาส	ประเภท	รายละเอียด
int tableNumber	แอตทริบิวต์	หมายเลขโต๊ะ
boolean occupied	แอตทริบิวต์	สถานะการใช้งาน
Order currentOrder	แอตทริบิวต์	ออร์เดอร์ปัจจุบัน
Staff assignedStaff	แอตทริบิวต์	พนักงานที่มอบหมาย
Table(int)	คอนสตรัคเตอร์	กำหนดค่าตามพารามิเตอร์
int getTableNumber()	เมธอด	คืนค่าหมายเลขโต๊ะ
boolean isOccupied()	เมธอด	ตรวจสอบสถานะโต๊ะ
Order getCurrentOrder()	เมธอด	คืนค่าออร์เดอร์ปัจจุบัน
void openTable(String)	เมธอด	เปิดโต๊ะ
void closeTable()	เมธอด	ปิดโต๊ะ
void printDetail()	เมธอด	พิมพ์สถานะโต๊ะ
void setAssignedStaff(Staff)	เมธอด	กำหนดพนักงานให้โต๊ะ
Staff getAssignedStaff()	เมธอด	คืนค่าพนักงานที่ถูกมอบหมาย

บทที่ 4 การออกแบบการทำงานของโปรแกรม

4.1 ภาพรวมการทำงานของโปรแกรม

โปรแกรม POS นี้มีการออกแบบขั้นตอนการทำงานสำหรับผู้ใช้งาน โดยเริ่มต้นโปรแกรมจะแสดงหน้าต่าง GUI พร้อมเมนูและปุ่มต่างๆ สำหรับการทำงานดังนี้

1. ผู้ใช้เลือกดูเมนูอาหารและเครื่องดื่ม
2. ผู้ใช้เลือกโต๊ะที่ต้องการเปิด (1-5)
3. ผู้ใช้กดปุ่ม Open Table เพื่อเปิดโต๊ะ
4. ผู้ใช้ป้อนรหัสสินค้า (Item ID) และกด Add Item เพื่อเพิ่มรายการ
5. เมื่อครบตามต้องการ ผู้ใช้ป้อนจำนวนเงินและกด Checkout เพื่อชำระเงิน
6. ระบบคำนวณยอดรวม ภาษี เงินทอน และแสดงใบเสร็จ

4.2 การออกแบบขั้นตอนการเปิดโต๊ะ

เมธอด openTable ในคลาส Table

1. รับพารามิเตอร์ orderId
2. สร้างออร์เดอร์ใหม่ด้วยรหัส orderId
3. กำหนด occupied = true
4. สิ้นสุด

4.3 การออกแบบขั้นตอนการคำนวณยอดรวม

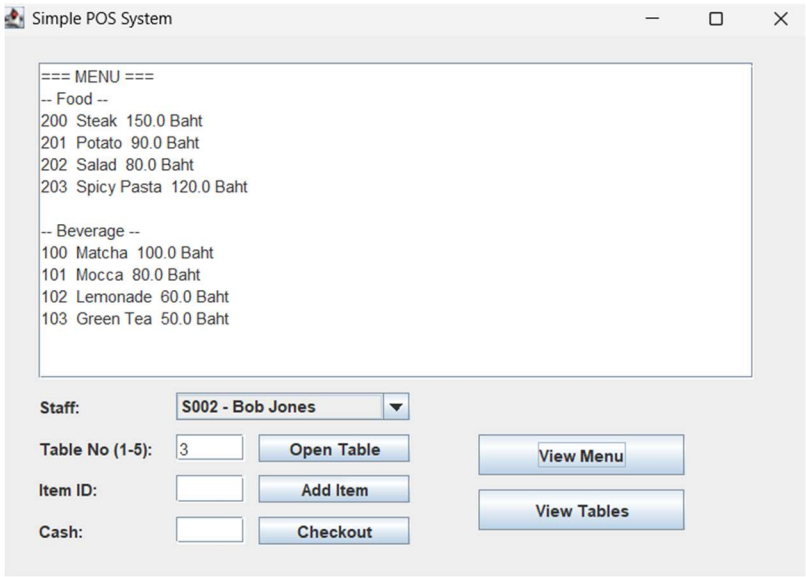
เมธอด calculateGrandTotal ในคลาส Payment

1. คำนวณยอดรวมย่อย (subtotal) จากออร์เดอร์
2. คำนวณภาษี = subtotal * taxRate
3. คืนค่า subtotal + ภาษี
4. สิ้นสุด

บทที่ 5 ผลลัพธ์ของโปรแกรม

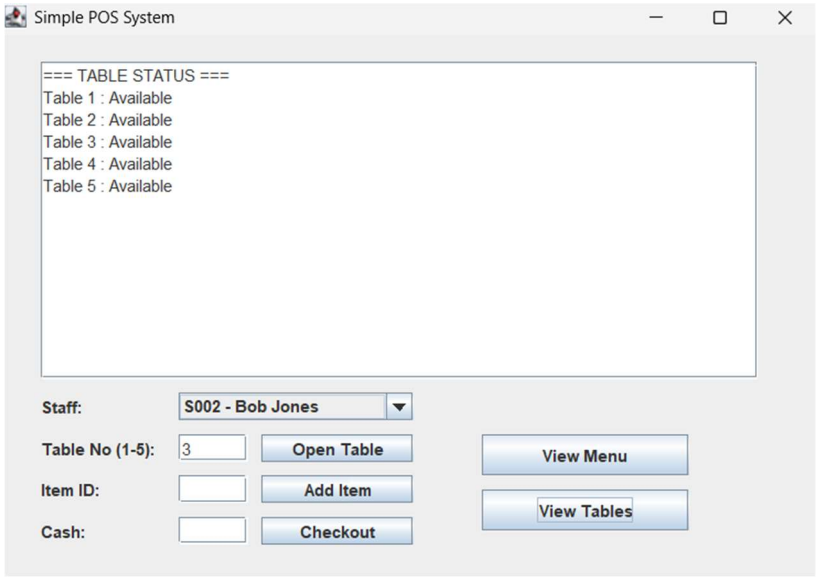
เมื่อเริ่มต้นรันโปรแกรม POS โปรแกรมจะแสดงหน้าต่าง GUI พร้อมปุ่มต่างๆ ดังนี้

• View Menu – แสดงรายการอาหารและเครื่องดื่ม



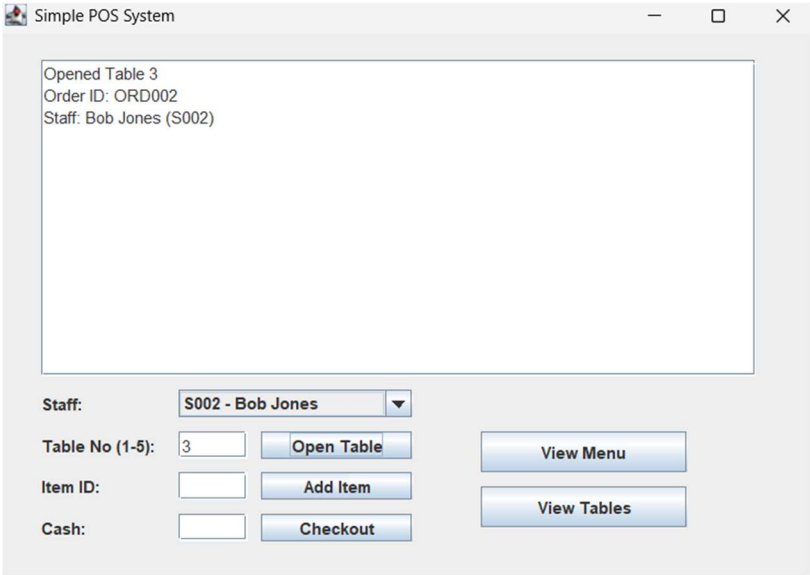
ภาพที่ 2 แสดงรายการอาหารและเครื่องดื่ม

• View Tables – แสดงสถานะโต๊ะทั้งหมด



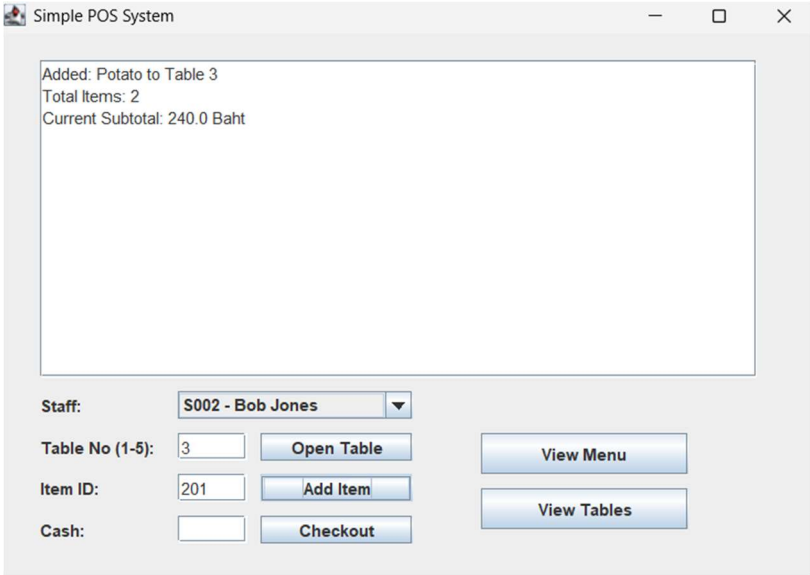
ภาพที่ 3 แสดงสถานะโต๊ะทั้งหมด

• Open Table – เปิดโต๊ะสำหรับรับลูกค้า



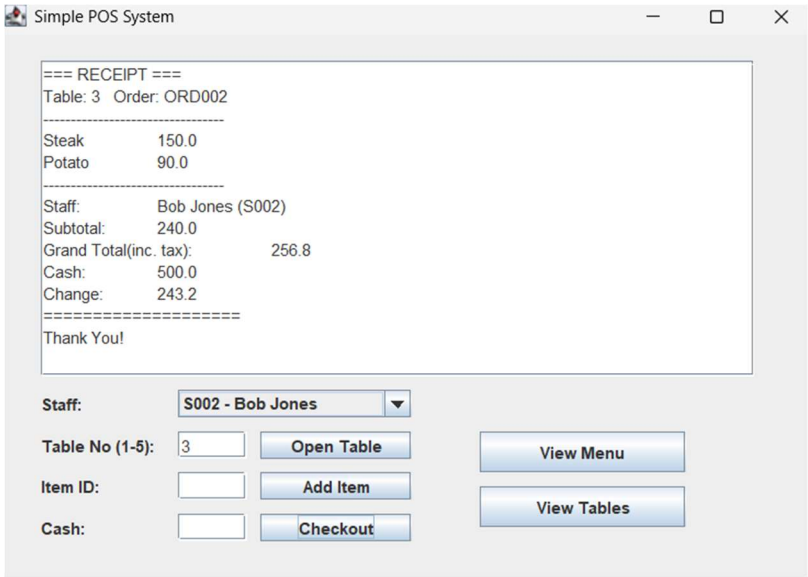
ภาพที่ 4 เปิดโต๊ะสำหรับรับลูกค้า

• Add Item – เพิ่มรายการลงในออร์เดอร์



ภาพที่ 5 เพิ่มรายการลงในออร์เดอร์

• Checkout – ชำระเงินและพิมพ์ใบเสร็จ



ภาพที่ 6 ชำระเงินและพิมพ์ใบเสร็จ

• Receipt – ตัวอย่างใบเสร็จที่แสดง

ภาพที่ 7 ตัวอย่างใบเสร็จ

บทที่ 6 สรุปผล

โปรแกรม POS ที่พัฒนาขึ้นนี้สามารถทำงานได้ตามวัตถุประสงค์ที่กำหนดไว้ โดยมีคุณสมบัติการสืบทอดจากคลาส Person ไปยัง Staff และ Customer การสืบทอดจากคลาส MenuItem ไปยัง Food และ Beverage การใช้อินเทอร์เฟซ Printable เพื่อกำหนดเมธอด printDetail ที่แต่ละคลาสต้อง implement และการใช้ Polymorphism ในการเรียกเมธอด printDetail ของแต่ละคลาส โปรแกรมสามารถจัดการโต๊ะ รับออร์เดอร์ คำนวณการชำระเงินรวมภาษีมูลค่าเพิ่ม 7% และแสดงใบเสร็จได้อย่างสมบูรณ์

บทที่ 7 ซอร์สโค้ด

ซอร์สโค้ดทั้งหมดประกอบด้วยไฟล์ต่างๆ ดังนี้

7.1 Printable.java – อินเทอร์เฟซสำหรับการพิมพ์รายละเอียด

```
// อินเตอร์เฟซสำหรับพิมพ์รายละเอียดของวัตถุ
public interface Printable {
    // พิมพ์รายละเอียดของวัตถุ
    void printDetail();
}
```

7.2 Person.java – คลาสพื้นฐานสำหรับบุคคล

```
// คลาสบุคคลพื้นฐาน มีชื่อและนามสกุล
public class Person implements Printable {

    // ชื่อ
    private String firstName;
    // นามสกุล
    private String lastName;

    // สร้าง Person ไม่ระบุชื่อ
    public Person() {
        this("", "");
    }

    // สร้าง Person ระบุชื่อและนามสกุล
    public Person(String firstName, String lastName) {
        this.firstName = firstName;
        this.lastName = lastName;
    }
```

```java
    // ตั้งชื่อและนามสกุลใหม่
    public void setFullName(String firstName, String lastName) {
        this.firstName = firstName;
        this.lastName = lastName;
    }

    // คืนชื่อ
    public String getFirstName() {
        return firstName;
    }

    // คืนค่านามสกุล
    public String getLastName() {
        return lastName;
    }

    // คืนชื่อเต็ม
    public String getFullName() {
        return firstName + " " + lastName;
    }

    // พิมพ์ชื่อเต็ม
    @Override
    public void printDetail() {
        System.out.println(getFullName());
    }
}
```

7.3 Staff.java – คลาสพนักงาน

```java
// คลาสพนักงาน extends Person มีรหัสพนักงาน
public class Staff extends Person {

    // รหัสพนักงาน
    private String staffId;

    // สร้างพนักงาน
    public Staff(String staffId, String firstName, String lastName) {
        super(firstName, lastName);
        this.staffId = staffId;
    }

    // คืนรหัสพนักงาน
    public String getStaffId() {
        return staffId;
```

```
    }

    // พิมพ์รายละเอียดพนักงาน
    @Override
    public void printDetail() {
        System.out.println("Staff ID: " + staffId + ", Name: " +
getFullName());
    }
}
```

7.4 Customer.java – คลาสลูกค้า

```
// คลาสลูกค้า extends Person มีรหัสลูกค้า
public class Customer extends Person {

    // รหัสลูกค้า
    private String customerId;

    // สร้างลูกค้า
    public Customer(String customerId, String firstName, String lastName)
{
        super(firstName, lastName);
        this.customerId = customerId;
    }

    // คืนรหัสลูกค้า
    public String getCustomerId() {
        return customerId;
    }

    // พิมพ์รายละเอียดลูกค้า
    @Override
    public void printDetail() {
        System.out.println("Customer ID: " + customerId + ", Name: " +
getFullName());
    }
}
```

7.5 MenuItem.java – คลาสพื้นฐานสำหรับรายการเมนู

```
import java.text.DecimalFormat;

// คลาสพื้นฐานรายการเมนู
public class MenuItem implements Printable {
```

```
    // รหัสประจำรายการ
    private String id;

    // ชื่อของรายการ
    private String name;

    // ราคา
    private double price;

    // สร้างเมนูใหม่
    public MenuItem(String id, String name, double price) {
        this.id = id;
        this.name = name;
        this.price = price;
    }

    // คืนรหัสรายการ
    public String getId() {
        return id;
    }

    // คืนชื่อรายการ
    public String getName() {
        return name;
    }

    // คืนราคา
    public double getPrice() {
        return price;
    }

    // พิมพ์รายละเอียดสั้นๆ
    @Override
    public void printDetail() {
        DecimalFormat df = new DecimalFormat("0.00");
        System.out.println(getId() + " " + getName() + " " +
df.format(getPrice()));
    }
}
```

7.6 Food.java – คลาสอาหาร

```
import java.text.DecimalFormat;
```

```java
// คลาสอาหาร extends MenuItem เก็บความเผ็ดและมังสวิรัติ
public class Food extends MenuItem {

    // ระดับความเผ็ด (0-100)
    private int spicinessLevel;

    // เป็นมังสวิรัติหรือไม่
    private boolean isVegan;

    // สร้างอาหารพร้อมตั้งค่าความเผ็ดและมังสวิรัติ
    public Food(String id, String name, double price, int spicinessLevel,
boolean isVegan) {
        super(id, name, price);
        this.spicinessLevel = spicinessLevel;
        this.isVegan = isVegan;
    }

    // คืนระดับความเผ็ด
    public int getSpicinessLevel() {
        return spicinessLevel;
    }

    // คืนว่าเป็นมังสวิรัติไหม
    public boolean isVegan() {
        return isVegan;
    }

    // พิมพ์รายละเอียดอาหาร
    @Override
    public void printDetail() {
        DecimalFormat df = new DecimalFormat("0.00");
        System.out.println(getId() + " " + getName() + " " +
df.format(getPrice())
                + " Spiciness:" + spicinessLevel + " Vegan:" + isVegan);
    }
}
```

7.7 Beverage.java – คลาสเครื่องดื่ม

```java
import java.text.DecimalFormat;

// คลาสสำหรับเครื่องดื่ม extends MenuItem
public class Beverage extends MenuItem {

    // ระดับความหวาน (0-100)
```

```
    private int sweetnessLevel;

    // เป็นเครื่องดื่มร้อนหรือไม่
    private boolean isHot;

    // สร้างเครื่องดื่มพร้อมตั้งค่าความหวานและสถานะร้อน
    public Beverage(String id, String name, double price, int
sweetnessLevel, boolean isHot) {
        super(id, name, price);
        this.sweetnessLevel = sweetnessLevel;
        this.isHot = isHot;
    }

    // คืนระดับความหวาน
    public int getSweetnessLevel() {
        return sweetnessLevel;
    }

    // คืนว่าร้อนไหม
    public boolean isHot() {
        return isHot;
    }

    // พิมพ์รายละเอียดเครื่องดื่ม
    @Override
    public void printDetail() {
        DecimalFormat df = new DecimalFormat("0.00");
        System.out.println(getId() + " " + getName() + " " +
df.format(getPrice())
                + " Sweetness:" + sweetnessLevel + " Hot:" + isHot);
    }
}
```

7.8 Order.java – คลาสออร์เดอร์

```
import java.text.DecimalFormat;
import java.util.ArrayList;
import java.util.List;

// แทนออร์เดอร์ลูกค้า เก็บรายการเมนูและคำนวณยอด
public class Order implements Printable {

    // รหัสออร์เดอร์
    private String orderId;
    // รายการเมนูในออร์เดอร์
```

```java
private List<MenuItem> items;

// สร้างออร์เดอร์ด้วยรหัส
public Order(String orderId) {
    this.orderId = orderId;
    this.items = new ArrayList<>();
}

// คืนรหัสออร์เดอร์
public String getOrderId() {
    return orderId;
}

// คืนรายการเมนูทั้งหมด
public List<MenuItem> getItems() {
    return items;
}

// คืนจำนวนรายการ
public int getItemCount() {
    return items.size();
}

// เพิ่มเมนูลงในออร์เดอร์
public void addItem(MenuItem item) {
    if (item != null) {
        items.add(item);
    }
}

// คำนวณยอดรวมย่อย
public double calculateTotal() {
    double total = 0;
    for (MenuItem item : items) {
        total += item.getPrice();
    }
    return total;
}

// พิมพ์รายละเอียดออร์เดอร์
@Override
public void printDetail() {
    DecimalFormat df = new DecimalFormat("0.00");
    System.out.println("Order ID: " + orderId);
    for (MenuItem item : items) {
        item.printDetail();
```

```
        }
        System.out.println("Subtotal: " + df.format(calculateTotal()));
    }
}
```

7.9 Payment.java – คลาสการชำระเงิน

```
import java.text.DecimalFormat;

// จัดการการชำระเงินสำหรับออร์เดอร์
public class Payment implements Printable {

    // ออร์เดอร์ที่กำลังชำระ
    private Order orderDetails;
    // อัตราภาษี เช่น 0.07
    private double taxRate;
    // ผู้ชำระเงิน (อาจเป็น null)
    private Person payer;

    // สร้าง Payment ด้วยออร์เดอร์และอัตราภาษี
    public Payment(Order orderDetails, double taxRate) {
        this(orderDetails, taxRate, null);
    }

    // สร้าง Payment พร้อมระบุผู้ชำระเงิน
    public Payment(Order orderDetails, double taxRate, Person payer) {
        this.orderDetails = orderDetails;
        this.taxRate = taxRate;
        this.payer = payer;
    }

    // คืนออร์เดอร์
    public Order getOrderDetails() {
        return orderDetails;
    }

    // คืนอัตราภาษี
    public double getTaxRate() {
        return taxRate;
    }

    // คืนผู้ชำระเงิน
    public Person getPayer() {
        return payer;
    }
```

```java
    // คำนวณยอดรวมทั้งหมดรวมภาษี
    public double calculateGrandTotal() {
        double subtotal = orderDetails.calculateTotal();
        return subtotal + (subtotal * taxRate);
    }

    // ประมวลผลการชำระและคืนเงินทอน
    public double processPayment(double cash) {
        double totalDue = calculateGrandTotal();
        if (cash >= totalDue) {
            return cash - totalDue;
        } else {
            System.out.println("Must pay more " + (totalDue - cash));
            return 0;
        }
    }

    // พิมพ์ใบเสร็จแบบง่าย
    public void printReceipt() {
        DecimalFormat df = new DecimalFormat("0.00");
        System.out.println("=== RECEIPT ===");
        if (payer != null) {
            System.out.println("Payer: " + payer.getFullName());
        }
        orderDetails.printDetail();
        System.out.println("Tax Rate: " + (taxRate * 100) + "%");
        System.out.println("Grand Total: " +
df.format(calculateGrandTotal()));
    }

    // พิมพ์รายละเอียด ตามอินเตอร์เฟซ Printable
    @Override
    public void printDetail() {
        printReceipt();
    }
}
```

7.10 Table.java – คลาสโต๊ะ

```java
// คลาสโต๊ะในร้านอาหาร
public class Table implements Printable {

    // หมายเลขโต๊ะ
    private int tableNumber;
```

```java
// สถานะการถูกใช้งาน
private boolean occupied;
// ออร์เดอร์ปัจจุบัน หรือ null
private Order currentOrder;
// พนักงานที่ถูกมอบหมาย หรือ null
private Staff assignedStaff;

// สร้างโต๊ะใหม่
public Table(int tableNumber) {
    this.tableNumber = tableNumber;
    this.occupied = false;
    this.currentOrder = null;
}

// คืนหมายเลขโต๊ะ
public int getTableNumber() {
    return tableNumber;
}

// ตรวจสอบว่าโต๊ะถูกใช้งานหรือไม่
public boolean isOccupied() {
    return occupied;
}

// คืนออร์เดอร์ปัจจุบัน
public Order getCurrentOrder() {
    return currentOrder;
}

// เปิดโต๊ะและสร้างออร์เดอร์ใหม่
public void openTable(String orderId) {
    this.currentOrder = new Order(orderId);
    this.occupied = true;
}

// ปิดโต๊ะและล้างออร์เดอร์
public void closeTable() {
    this.currentOrder = null;
    this.occupied = false;
    this.assignedStaff = null;
}

// พิมพ์สถานะของโต๊ะ
@Override
public void printDetail() {
    if (occupied) {
```

```
            String staffInfo = (assignedStaff != null)
                    ? assignedStaff.getStaffId() + " " +
assignedStaff.getFullName()
                    : "Unassigned";
            System.out.println("Table " + tableNumber + " : Occupied
(Order: " + currentOrder.getOrderId()
                    + ", Staff: " + staffInfo + ")");
        } else {
            System.out.println("Table " + tableNumber + " : Available");
        }
    }

    // กำหนดพนักงานให้โต๊ะ
    public void setAssignedStaff(Staff staff) {
        this.assignedStaff = staff;
    }

    // คืนพนักงานที่ถูกมอบหมาย
    public Staff getAssignedStaff() {
        return assignedStaff;
    }
}
```

7.11 SimplePOS_GUI.java - คลาส GUI หลัก

```
import javax.swing.*;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import java.util.List;

// GUI ง่ายสำหรับระบบ POS
public class SimplePOS_GUI {

    // เครื่องดื่มตัวอย่าง
    static Beverage[] beverages = {
            new Beverage("100", "Matcha", 100, 100, true),
            new Beverage("101", "Mocca", 80, 50, true),
            new Beverage("102", "Lemonade", 60, 75, false),
            new Beverage("103", "Green Tea", 50, 30, true),
            new Beverage("104", "Thai Tea", 60, 75, false),
            new Beverage("105", "Taiwan Tea", 75, 75, false)
    };

    // อาหารตัวอย่าง
    static Food[] foods = {
```

```
            new Food("200", "Steak", 150, 50, false),
            new Food("201", "Potato", 90, 50, false),
            new Food("202", "Salad", 80, 0, true),
            new Food("203", "Spicy Pasta", 120, 80, false),
            new Food("204", "Pat Thai", 160, 60, false),
            new Food("205", "Salmon Kraprao", 350, 60, false)
    };

    // โต๊ะในร้านอาหาร
    static Table[] tables = new Table[5];

    // ตัวนับสร้างรหัสออร์เดอร์
    static int orderCounter = 1;

    // พนักงานตัวอย่าง
    static Staff[] staffMembers = {
            new Staff("S001", "Alice", "Smith"),
            new Staff("S002", "Bob", "Jones"),
            new Staff("S003", "Carol", "Davis")
    };

    // เมธอดหลักสำหรับสร้าง GUI และ event listener
    public static void main(String[] args) {
        // สร้างโต๊ะ
        for (int i = 0; i < tables.length; i++) {
            tables[i] = new Table(i + 1);
        }

        // ตั้งค่า GUI
        JFrame fr = new JFrame();
        fr.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        fr.setBounds(100, 100, 600, 500);
        fr.setTitle("Simple POS System");
        fr.getContentPane().setLayout(null);

        JTextArea display = new JTextArea();
        display.setEditable(false);
        JScrollPane scroll = new JScrollPane(display);
        scroll.setBounds(30, 20, 520, 230);
        fr.getContentPane().add(scroll);
        display.setText("Welcome to POS System\nClick 'View Menu' or 'View
Tables' to start.");

        JLabel lblStaff = new JLabel("Staff:");
        lblStaff.setBounds(30, 260, 100, 20);
        fr.getContentPane().add(lblStaff);
```

```
        String[] staffOptions = new String[staffMembers.length];
        for (int i = 0; i < staffMembers.length; i++) {
            staffOptions[i] = staffMembers[i].getStaffId() + " - " +
staffMembers[i].getFullName();
        }
        JComboBox<String> cmbStaff = new JComboBox<>(staffOptions);
        cmbStaff.setBounds(130, 260, 170, 20);
        fr.getContentPane().add(cmbStaff);

        JLabel lblTable = new JLabel("Table No (1-5):");
        lblTable.setBounds(30, 290, 100, 20);
        fr.getContentPane().add(lblTable);

        JTextField txtTable = new JTextField();
        txtTable.setBounds(130, 290, 50, 20);
        fr.getContentPane().add(txtTable);

        JButton btnOpen = new JButton("Open Table");
        btnOpen.setBounds(190, 290, 110, 20);
        fr.getContentPane().add(btnOpen);

        JLabel lblItem = new JLabel("Item ID:");
        lblItem.setBounds(30, 320, 100, 20);
        fr.getContentPane().add(lblItem);

        JTextField txtItem = new JTextField();
        txtItem.setBounds(130, 320, 50, 20);
        fr.getContentPane().add(txtItem);

        JButton btnAdd = new JButton("Add Item");
        btnAdd.setBounds(190, 320, 110, 20);
        fr.getContentPane().add(btnAdd);

        JLabel lblCash = new JLabel("Cash:");
        lblCash.setBounds(30, 350, 100, 20);
        fr.getContentPane().add(lblCash);

        JTextField txtCash = new JTextField();
        txtCash.setBounds(130, 350, 50, 20);
        fr.getContentPane().add(txtCash);

        JButton btnCheckout = new JButton("Checkout");
        btnCheckout.setBounds(190, 350, 110, 20);
        fr.getContentPane().add(btnCheckout);
```

```
        JButton btnViewMenu = new JButton("View Menu");
        btnViewMenu.setBounds(350, 290, 150, 30);
        fr.getContentPane().add(btnViewMenu);

        JButton btnViewTables = new JButton("View Tables");
        btnViewTables.setBounds(350, 330, 150, 30);
        fr.getContentPane().add(btnViewTables);

        // ตั้งค่า event listener
        btnViewMenu.addActionListener(new ActionListener() {
            public void actionPerformed(ActionEvent arg0) {
                display.setText("=== MENU ===\n");
                display.append("-- Food --\n");
                for (int i = 0; i < foods.length; i++) {
                    display.append(foods[i].getId() + "  " +
foods[i].getName() + "  "
                            + foods[i].getPrice() + " Baht\n");
                }
                display.append("\n-- Beverage --\n");
                for (int i = 0; i < beverages.length; i++) {
                    display.append(beverages[i].getId() + "  " +
beverages[i].getName() + "  "
                            + beverages[i].getPrice() + " Baht\n");
                }
            }
        });

        btnViewTables.addActionListener(new ActionListener() {
            public void actionPerformed(ActionEvent arg0) {
                display.setText("=== TABLE STATUS ===\n");
                for (int i = 0; i < tables.length; i++) {
                    Table t = tables[i];
                    if (t.isOccupied()) {
                        String staffName = (t.getAssignedStaff() != null)
                                ? t.getAssignedStaff().getFullName() + "
(" + t.getAssignedStaff().getStaffId() + ")"
                                : "Unassigned";
                        display.append("Table " + t.getTableNumber() + " :
Occupied (Order: "
                                + t.getCurrentOrder().getOrderId() + ",
Staff: " + staffName + ")\n");
                    } else {
                        display.append("Table " + t.getTableNumber() + " :
Available\n");
                    }
                }
```

```
            }
        });

        btnOpen.addActionListener(new ActionListener() {
            public void actionPerformed(ActionEvent arg0) {
                try {
                    int tNum = Integer.parseInt(txtTable.getText());
                    if (tNum >= 1 && tNum <= tables.length) {
                        Table t = tables[tNum - 1];
                        if (t.isOccupied()) {
                            JOptionPane.showMessageDialog(fr, "Table " +
tNum + " is already occupied.");
                        } else {
                            String orderId = "ORD" + String.format("%03d",
orderCounter++);
                            t.openTable(orderId);
                            int staffIndex = cmbStaff.getSelectedIndex();
                            Staff selectedStaff =
staffMembers[staffIndex];
                            t.setAssignedStaff(selectedStaff);
                            display.setText("Opened Table " + tNum +
"\nOrder ID: " + orderId
                                    + "\nStaff: " +
selectedStaff.getFullName() + " (" + selectedStaff.getStaffId() + ")");
                        }
                    } else {
                        JOptionPane.showMessageDialog(fr, "Table must be
1-" + tables.length);
                    }
                } catch (Exception e) {
                    JOptionPane.showMessageDialog(fr, "Invalid Table
Number");
                }
            }
        });

        btnAdd.addActionListener(new ActionListener() {
            public void actionPerformed(ActionEvent arg0) {
                try {
                    int tNum = Integer.parseInt(txtTable.getText());
                    String itemId = txtItem.getText();
                    if (tNum >= 1 && tNum <= tables.length) {
                        Table t = tables[tNum - 1];
                        if (!t.isOccupied()) {
                            JOptionPane.showMessageDialog(fr, "Table is
not open. Please open table first.");
```

```
                            return;
                        }
                        MenuItem found = null;
                        for (int i = 0; i < foods.length; i++) {
                            if (foods[i].getId().equals(itemId)) {
                                found = foods[i];
                                break;
                            }
                        }
                        if (found == null) {
                            for (int i = 0; i < beverages.length; i++) {
                                if (beverages[i].getId().equals(itemId)) {
                                    found = beverages[i];
                                    break;
                                }
                            }
                        }
                        if (found != null) {
                            t.getCurrentOrder().addItem(found);
                            display.setText("Added: " + found.getName() +
" to Table " + tNum + "\n");
                            display.append("Total Items: " +
t.getCurrentOrder().getItemCount() + "\n");
                            display.append("Current Subtotal: " +
t.getCurrentOrder().calculateTotal() + " Baht");
                        } else {
                            JOptionPane.showMessageDialog(fr, "Item ID not
found!");
                        }
                    }
                } catch (Exception e) {
                    JOptionPane.showMessageDialog(fr, "Invalid Input");
                }
            }
        });

        btnCheckout.addActionListener(new ActionListener() {
            public void actionPerformed(ActionEvent arg0) {
                try {
                    int tNum = Integer.parseInt(txtTable.getText());
                    double cash = Double.parseDouble(txtCash.getText());
                    if (tNum >= 1 && tNum <= tables.length) {
                        Table t = tables[tNum - 1];
                        if (!t.isOccupied()) {
                            JOptionPane.showMessageDialog(fr, "Table is
not open.");
```

```
                            return;
                        }
                        if (t.getCurrentOrder().getItemCount() == 0) {
                            JOptionPane.showMessageDialog(fr, "No items in
order.");
                            return;
                        }
                        Payment p = new Payment(t.getCurrentOrder(),
0.07);
                        double grandTotal = p.calculateGrandTotal();
                        if (cash >= grandTotal) {
                            double change = cash - grandTotal;
                            display.setText("=== RECEIPT ===\n");
                            display.append("Table: " + tNum + "   Order: "
+ t.getCurrentOrder().getOrderId() + "\n");
                            display.append("------------------------------
---\n");
                            List<MenuItem> orderItems =
t.getCurrentOrder().getItems();
                            for (MenuItem item : orderItems) {
                                display.append(item.getName() + " \t" +
item.getPrice() + "\n");
                            }
                            display.append("------------------------------
---\n");
                            if (t.getAssignedStaff() != null) {
                                display.append("Staff: \t" +
t.getAssignedStaff().getFullName()
                                            + " (" +
t.getAssignedStaff().getStaffId() + ")\n");
                            }
                            display.append("Subtotal: \t" +
t.getCurrentOrder().calculateTotal() + "\n");
                            display.append("Grand Total(inc. tax): \t" +
grandTotal + "\n");
                            display.append("Cash: \t" + cash + "\n");
                            display.append("Change: \t" + change + "\n");
                            display.append("====================\nThank
You!");
                            t.closeTable();
                            txtItem.setText("");
                            txtCash.setText("");
                        } else {
                            JOptionPane.showMessageDialog(fr, "Not enough
cash! Need " + (grandTotal - cash) + " more.");
                        }
```

```
                    }
                } catch (Exception e) {
                    JOptionPane.showMessageDialog(fr, "Invalid Input /
Missing Cash");
                }
            }
        });
        fr.setVisible(true);
    }
}
```